

ROS2 디버깅 실습

rqt_graph · rqt_plot · rqt_console

cchyun@gmail.com

Part 1. rqt 디버깅 도구

- 실습 목표

- 런치 파일로 전체 시스템을 한 번에 기동
- `rqt` 도구를 활용하여 노드 간의 데이터 흐름과 상태를 실시간 모니터링 및 진단

- 실습 대상 시스템

- Day 6 ~ Day 8에서 개발한 노드들을 통합
- 카메라 퍼블리시 노드 + YOLO 탐지 노드 + 커스텀 메시지

rqt란 무엇인가?

- rqt는 ROS 2에서 시스템 상태를 관리하고 상호작용하기 위한 GUI 프레임워크
 - 별도 코드 없이 실행 중인 시스템을 **눈으로 보면서** 분석 가능
 - 플러그인 방식으로 다양한 도구를 하나의 창에서 사용
- 주요 rqt 도구 목록

도구	역할
rqt_graph	노드-토픽 연결 구조 시각화
rqt_plot	숫자 데이터 실시간 그래프
rqt_console	로그 메시지 중앙 모니터링
rqt_image_view	카메라 이미지 실시간 출력

- 현재 실행 중인 모든 노드와 토픽 흐름을 "ROS 그래프" 형태로 시각화
 - **Node graph**: 시스템의 전체 구조를 한눈에 파악
 - **Topic filter**: 특정 토픽만 보거나 숨길 수 있어 복잡한 시스템에서 원하는 부분만 집중 분석
 - **Hide 옵션**: 디버깅에 불필요한 노드나 토픽을 숨겨 그래프를 단순화

```
# 새 터미널에서 실행  
rqt_graph
```

- 토픽에 포함된 숫자 데이터(Numeric fields)를 2D 그래프로 실시간 플롯
 - 필드 지정 방법
 - `topic_name/field_name` 형식으로 플롯할 대상을 입력
 - 예: `/detection_result/score`
 - 스케일 조정
 - 데이터의 변화 폭에 맞춰 그래프 축을 자동 또는 수동으로 조정
 - 탐지 신뢰도 트렌드 분석에 활용

```
ros2 run rqt_plot rqt_plot /image_visualized/header/stamp/nanosec
```

- TurtleBot topic 시각화

```
sudo apt install ros-humble-topic-tools
```

```
ros2 run turtlesim turtle_teleop_key
```

```
ros2 run topic_tools transform /turtle1/cmd_vel /turtle_linear_x std_msgs/msg/Float32 'std_msgs.msg.Float32(data=m.linear.x)' --import std_msgs
```

```
ros2 run topic_tools transform /turtle1/cmd_vel /turtle_angular_z std_msgs/msg/Float32 'std_msgs.msg.Float32(data=m.angular.z)' --import std_msgs
```

```
ros2 run rqt_plot rqt_plot /turtle_linear_x/data /turtle_angular_z/data
```

- 여러 노드에서 발생하는 로그 메시지를 한곳에 모아 보여주는 중앙 로그 뷰어
 - 로그 레벨 필터링
 - 메시지의 심각도(Severity)에 따라 필터링
 - 특정 수준 이상의 문제만 집중적으로 확인 가능

색상	레벨	의미
흰색	DEBUG / INFO	정상 동작 정보
주황색	WARN	경고 — 기능은 작동
빨간색	ERROR / FATAL	오류 — 즉시 확인 필요

```
ros2 run rqt_console rqt_console
```


- 카메라 노드가 퍼블리시하는 이미지 데이터를 실시간으로 화면에 출력
 - 드롭다운 메뉴에서 구독할 토픽 선택
 - 카메라 연결 여부와 이미지 품질을 즉시 확인 가능

```
ros2 run rqt_image_view rqt_image_view
```

- 또는 rqt 통합 창에서 플러그인으로 추가

```
rqt  
# 상단 메뉴 → Plugins → Visualization → Image View
```

Part 2. 단계별 실습 시나리오

1단계 — 런치 파일로 전체 시스템 기동

- Day 6 ~ 8에서 만든 노드들을 하나의 런치 파일(`.py`)로 묶어 실행
 - 런치 파일은 복잡한 시스템의 설정을 자동화하고 여러 노드를 동시 실행하는 데 필수

```
ros2 launch <패키지_이름> <통합_런치파일_이름>.launch.py
```

- 기동 확인 포인트
 - 터미널에 오류(ERROR) 없이 노드 시작 로그가 출력되는지 확인
 - `ros2 node list`로 의도한 노드들이 모두 실행 중인지 확인

```
ros2 node list  
ros2 topic list
```

2단계 — rqt_graph로 연결 확인 및 문제 진단

- 새 터미널에서 `rqt_graph`를 실행하여 노드들이 의도대로 연결되었는지 확인
 - 데이터 흐름 점검: 카메라 노드 → YOLO 노드 → 커스텀 메시지 출력 노드
 - 선을 따라가며 토픽 연결이 끊기지 않았는지 확인
- 문제 진단 방법

증상	원인	조치
연결 선이 없음	토픽 이름 불일치	<code>ros2 topic list</code> 로 이름 확인
노드 이름이 빨간색	노드 크래시 또는 오류	<code>rqt_console</code> 에서 ERROR 로그 확인
노드가 고립됨	리매핑 오류	런치 파일의 remapping 설정 점검

3단계 — rqt_plot으로 탐지 신뢰도 모니터링

- `rqt_plot`은 숫자형 토픽만 시각화 가능 → 탐지 통계를 별도 노드로 발행 필요
 - YOLO 결과(`detection_result`)를 구독해 숫자 통계를 새 토픽으로 발행하는 노드 추가
- `rqt_plot`에서 모니터링할 토픽 추가

토픽	내용
<code>/detection_count/data</code>	프레임당 탐지 개수
<code>/avg_confidence/data</code>	평균 신뢰도 (0.0 ~ 1.0)

입력창에 토픽 경로 입력 후 **+** 버튼을 눌러 그래프에 추가

- 관찰 실험
 - 탐지 대상이 가려질 때 → `avg_confidence` 수치 하락 관찰
 - 탐지 대상이 멀어질 때 → `avg_confidence` & `detection_count` 동시 변화 관찰
 - 탐지 대상이 없을 때 → 두 값이 모두 0으로 수렴하는지 확인

4단계 — rqt_console로 로그 레벨 필터링

- `rqt_console`을 실행한 상태에서 시스템을 관찰
 - YOLO 노드에서 탐지 대상이 일정 시간 나타나지 않을 때 `WARN` 로그가 발생하도록 설정
 - 콘솔 창에서 주황색 메시지(`WARN`)가 나타나는지 확인
- 로그 레벨 필터 설정 실습

[rqt_console 화면]

1. 상단의 "Exclude messages" 또는 "Filter" 패널 사용
2. "INFO" 레벨을 숨김(Hide) 처리
3. "WARN" 이상의 메시지만 표시되도록 설정

- 노이즈를 줄이고 진짜 문제만 빠르게 파악하는 실전 디버깅 기법

5단계 — 의도적 버그 심기 → rqt로 원인 찾기

- 버그 주입 → 발견 → 수정의 디버깅 전 과정 체험

- Step 1: 버그 심기

- 런치 파일에서 YOLO 노드가 구독하는 토픽 이름을 의도적으로 틀리게 수정 (Remapping 오류)

```
# 런치 파일에서 토픽 이름을 잘못 설정  
remappings=[('/camera/image_raw', '/wrong_topic_name')]
```

- Step 2: rqt로 원인 찾기

- `rqt_graph` 새로고침 → YOLO 노드가 고립되어 있는지 확인
- `rqt_console` → "메시지가 들어오지 않는다"는 경고 로그 확인

- Step 3: 수정 후 재확인

- 토픽 이름 수정 → 런치 재실행 → `rqt_graph` 로 연결 복구 확인

Part 3. `get_logger()` 로그 레벨

- Python 노드(`rcipy.node.Node`) 내부에서 `self.get_logger()` 로 상황에 맞는 로그 레벨 출력

레벨	메소드	설명
DEBUG	<code>self.get_logger().debug()</code>	상세한 단계별 과정 기록 (기본적으로 숨겨짐)
INFO	<code>self.get_logger().info()</code>	정상 실행 중임을 알리는 일반 정보
WARN	<code>self.get_logger().warn()</code>	이상 징후 경고 — 기능은 작동 중
ERROR	<code>self.get_logger().error()</code>	기능 동작을 방해하는 심각한 문제
FATAL	<code>self.get_logger().fatal()</code>	시스템 중단이 필요한 치명적 오류

- 실제 노드에서 로그 레벨을 적절히 사용하는 예시

```
class YoloDetectorNode(Node):
    def __init__(self):
        super().__init__('yolo_detector')
        self.get_logger().info('YOLO 탐지 노드 시작') # 정상 시작

    def detect_callback(self, msg):
        result = self.run_yolo(msg)

        if result is None:
            # 탐지 결과 없음 - 경고 수준
            self.get_logger().warn('탐지 결과 없음: 대상이 보이지 않습니다')
        elif result.score < 0.3:
            # 낮은 신뢰도 - 정보 수준
            self.get_logger().info(f'낮은 신뢰도: {result.score:.2f}')
        else:
            self.get_logger().debug(f'탐지 성공: {result.score:.2f}')
```

Part 4. P3 마무리

- Day 6: ROS2 기초 및 토픽 통신

- 노드(Nodes): 단일 목적을 수행하는 모듈식 프로세스 — 로봇 시스템의 최소 실행 단위
- 토픽(Topics): 연속적인 데이터 스트림을 위한 다대다 퍼블리시-서브스크라이브 메커니즘
- rclpy: Python으로 ROS2 통신 기능을 구현하는 공식 클라이언트 라이브러리

- Day 7: 서비스, 액션 및 커스텀 메시지

- 서비스(Services): 짧게 끝나는 작업을 위한 일대일 요청-응답 패턴
- 액션(Actions): 목표 설정·피드백·결과 제공이 가능한 장시간 실행 작업용 통신 방식
- 커스텀 인터페이스: `.msg`, `.srv`, `.action` 파일로 사용자 정의 데이터 구조 생성



- Day 8: 통합 기동 및 디버깅 도구
 - 런치 파일: 여러 노드 동시 실행 · 파라미터 설정 · 토픽 리매핑 자동화
 - 파라미터: 코드 수정 없이 런타임에 노드 설정을 변경하는 노드별 설정값
 - rqt 도구: `rqt_graph`, `rqt_console` 등 GUI 기반 인트로스펙션 도구 모음